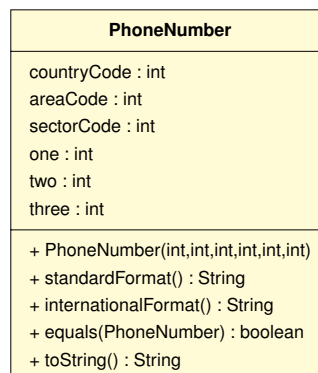


Exercice 1 : Manipulation de tables associatives

L'objectif de cet exercice est de développer un répertoire téléphonique bidirectionnel. Un tel répertoire permet de trouver un numéro à partir d'un nom ou un nom à partir d'un numéro. On va restreindre le cadre d'utilisation aux personnels de l'université.

Vous disposez d'une classe `PhoneNumber` (à partir d'une archive `jar`) : elle avait été développée pour un premier travail dirigé : elle appartient au paquetage `td00.td01`. Comme elle convient à peu près à ce dont nous avons besoin, nous allons la recycler. Elle répond aux spécifications UML suivantes :



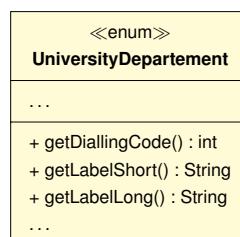
En raison du contexte applicatif restreint, cette classe est beaucoup plus générique que ce que nous avons besoin : le code du pays est toujours le même (ici `country=33`), tout comme celui de la zone (ici `area=03`) ou du secteur (ici `sector=20`) car l'université ne va pas déménager... Reste 0+le code de la composante (sur 1 digit) et le numéro du poste (les 4 derniers digits). On va donc restreindre son utilisation.

Un numéro de l'université peut s'utiliser soit en interne et on ne compose que les 5 derniers digits (le code de la composante suivi du numéro du poste), soit en externe et il faut alors composer tous les digits.

Q1. Intéressons d'abord à la structure de l'université. Elle est organisée en 8 composantes, chacune bénéficiant d'un indicatif propre (allant de 1 à 8 selon la composante).

Q1.1. Récupérez le squelette de code du fichier `UniversityDepartement.java` et intégrez-le à votre projet.

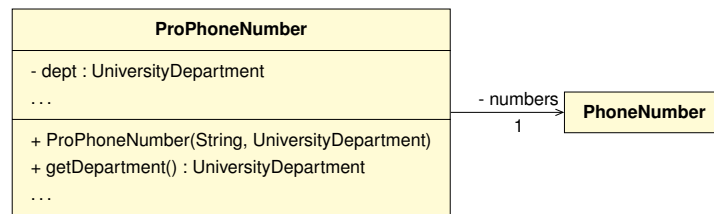
Q2. Après avoir déduit la structure et les éléments manquants aux spécifications UML, complétez l'`enum` en sachant qu'on doit pouvoir récupérer l'indicatif de la composante ainsi que le nom soit en version courte (le sigle), soit en version longue.



Q3. C'est parti pour le recyclage ! On souhaite bénéficier d'une classe `ProPhoneNumber` répondant spécifiquement à nos besoins.

Q3.1. Récupérez et intégrez à votre projet la classe `PhoneNumber` via son archive.

Q3.2. Écrivez la classe `ProPhoneNumber` **encapsulant** la classe `PhoneNumber`, tout en respectant les spécifications UML suivantes :



Notez que le constructeur prend en paramètre les 4 derniers digits du numéro de téléphone sous forme d'une chaîne de caractères ainsi que le département auquel est associé le numéro.

Q3.3. Munissez cette classe des méthodes `internToString()` et `externToString()` qui fournissent un affichage respectivement pour l'utilisation interne et externe. Par exemple, le poste 1234 de l'IUT s'affichera :

- 31234 (IUT) en interne
- +33.3.20.03.12.34 (Institut Universitaire de Technologies) en externe

```
String internToString()
String externToString()
```

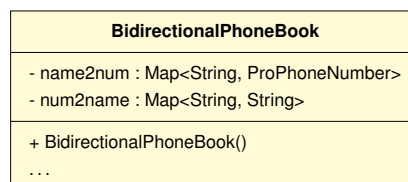
Q4. Ajoutez à votre classe une méthode `equals(...)` classique, prenant un paramètre de type `Object`, et une surcharge prenant spécifiquement un paramètre de type `String` correspondant au numéro du poste (les 4 derniers digits).

```
boolean equals(Object obj)
boolean equals(String fourDigits)
```

Q5. Intégrez la classe de tests `ProPhoneNumberTest` à votre projet et confirmez votre implémentation en validant tous les tests fournis.

Q6. Intéressons-nous au répertoire bidirectionnel. Il va reposer sur deux tables associatives de type `HashMap` gérées en parallèles. Par exemple, l'ajout d'une entrée dans le répertoire bidirectionnel mène à l'ajout d'une entrée dans chacune des tables associatives. La première table `name2num` associe un nom d'utilisateur à un numéro de téléphone professionnel (`ProPhoneNumber`) alors que la seconde associera le numéro interne (5 derniers digits, sous forme d'une chaîne de caractères) à un nom d'utilisateur.

Q6.1. Écrivez la classe `BidirectionalPhoneBook`, munie d'un constructeur sans paramètre, répondant aux spécifications suivantes :



Q6.2. Ajoutez une méthode `getNbEntries()` qui retourne le nombre d'entrées présentes dans le répertoire bidirectionnel.

```
int getNbEntries()
```

Q6.3. Écrivez une méthode `alreadyRegistered(...)` qui détermine si une chaîne passée en paramètre est soit un nom déjà utilisé, soit 5 digits déjà enregistrés (utilisé comme clé de l'une ou l'autre tables associatives).

```
boolean alreadyRegistered(String s)
```

Q6.4. Écrivez une méthode `add(...)` qui ajoute une entrée au répertoire bidirectionnel à partir d'un nom, d'une composante de l'université et du numéro de poste (4 digits). La méthode doit retourner un booléen selon le succès ou l'échec de l'opération.

```
boolean add(String name, UniversityDepartment dept, String fourDigits)
```

Q6.5. Ajoutez les méthodes de recherche d'un numéro à partir nom (`getProPhoneNumberFromName(...)`) et d'un nom à partir des 5 digits servant de clé (`getNameFromFiveDigits(...)`).

```
ProPhoneNumber getProPhoneNumberFromName(String name)
String getNameFromFiveDigits(String fiveDigits)
```

Q6.6. On souhaite pouvoir générer des "listes" téléphoniques pour un usage interne qui soient générales (incluant tous les numéros, regroupés par composante) ou pour une composante donnée. Le résultat commence toujours par le nom (en version longue) de la composante, puis sur chacune des lignes suivantes les noms et numéros (en version interne). L'ensemble se répète si plusieurs composantes sont concernées. Écrivez les méthodes `listing(...)` nécessaires à cela.

```
String listing(UniversityDepartment dept)
String listing()
```

Q7. Intégrez la classe de tests `BidirectionalPhoneBookTest` à votre projet et confirmez votre implémentation en validant tous les tests fournis.

Q8. Réalisez un `commit` associé au message : "tp00-07::exo-répertoireBidirectionnel". Soumettez ensuite l'ensemble des `commit` au dépôt.

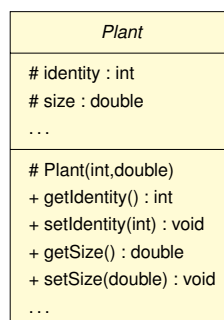
Exercice 2 : Exploitation horticole

On souhaite gérer une exploitation horticole (*horticultural Garden*), qui regroupe différents types de cultures : les rosiers (*rose*), les tulipes (*tulip*) et les arbustes (*shrub*). La majorité de ces plantes ont les mêmes règles de gestion, basées sur des valeurs propres à chaque espèce.

Toutes les plantes sont caractérisées par un numéro d'identification (lot) ainsi qu'une taille. Chacune des familles de plantes est caractérisée par un prix unitaire (`pricePerUnit`) ainsi qu'un seuil de maturité (`harvestThreshold`) au-delà duquel la plante peut être vendue. Le prix unitaire, tout comme le seuil de maturité, peut varier au cours du temps, mais la valeur est la même pour toutes les plantes de la même espèce.

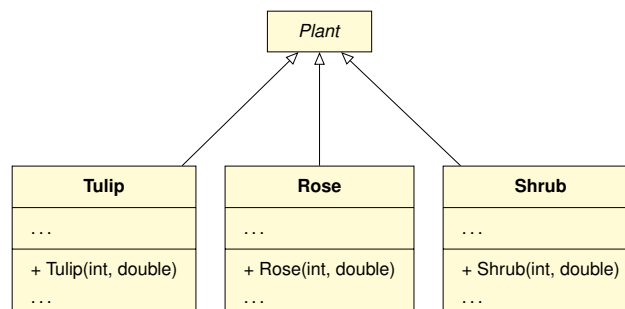
Q1. Intéressons-nous aux plantes dans un premier temps.

Q1.1. Afin de minimiser la redondance de code, un maximum de caractéristiques communes vont être extraites dans une classe abstraite. Écrivez la classe `Plant` répondant aux spécifications UML suivantes :



Q1.2. Écrivez les classes `Rose`, `Tulip` et `Shrub`, dérivant de `Plant`, en sachant que :

- un rosier rapporte 2 €/unité et peut être vendu à partir de 50 cm
- une tulipe rapporte 1 €/unité et peut être vendue à partir de 30 cm
- un arbuste rapporte 5 €/unité et peut être vendu à partir de 100 cm



Q1.3. En réfléchissant bien à la position/décomposition du code, ajoutez-la(les) méthode(s) nécessaire(s) à l'affichage afin que chaque volaille s'affiche sur le modèle suivant : `<type> [<identity>, <size>]`.

Q1.4. En sachant que la formule de calcul du prix est la même quel que soit le type de plante ($\text{cm} \times \text{€/cm}$), écrivez la méthode `getPrice()`, retournant le prix qu'on peut espérer d'une plante. Veillez à bien réfléchir à la décomposition du code.

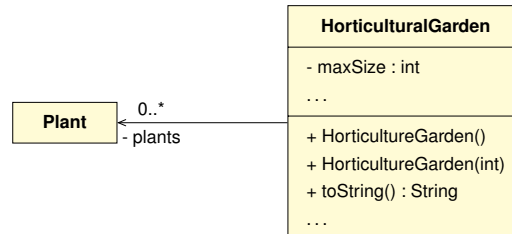
```
double getPrice()
```

Q1.5. Ajoutez les méthodes `setPricePerUnit` et `setHarvestThreshold` permettant respectivement de modifier le prix unitaire et le seuil de maturité pour chaque espèce.

Q1.6. Validez votre implémentation à l'aide des classes de tests fournies.

Q2. Intéressons-nous maintenant à l'exploitation horticole. Chaque exploitation peut contenir différents types de plantes et dispose d'une taille maximale (2000 par défaut) qui dépend de la superficie de culture disponible. Nous n'utilisons qu'une seule structure de données de type "Collection".

Q2.1. Écrivez la classe `HorticulturalGarden` répondant aux spécifications UML suivantes :



Q2.2. Ajoutez une méthode `add` permettant d'ajouter une plante si la capacité maximale n'est pas atteinte.

```
void add(Plant plant)
```

Q2.3. Ajoutez les méthodes `getGardenSize` et `getNbRose` qui retournent respectivement le nombre total de plantes et le nombre de roses.

```
int getGardenSize()
int getNbRose()
```

Q2.4. Ajoutez une méthode `search` permettant de retrouver une plante à partir de son identifiant.

```
Plant search(int identity)
```

Q2.5. Écrivez une méthode `getRose` retournant le i-ème rosier, ou `null` le cas échéant.

```
Rose getRose(int i)
```

Q2.6. Ajoutez une méthode `updateSize` permettant de modifier la taille d'une plante.

```
void updateSize(int identity, double size)
```

Q3. Parlons rendement maintenant !

Q3.1. Ajoutez une méthode `potentialProfit` permettant de calculer le gain potentiel si toutes les plantes arrivées à maturité sont vendues.

```
double potentialProfit()
```

Q3.2. Ajoutez une méthode `harvest()` qui retire les plantes prêtes à être vendues et les retourne.

```
List<Plant> harvest()
```

Q3.3. Récupérez la classe de tests `HorticulturalGardenTest` Validez votre implémentation.

Q3.4. Certaines plantes (rosiers et arbustes) peuvent bénéficier de techniques de culture premium, qui accélèrent la croissance. À l'aide d'une interface `IPremiumCare`, écrivez une méthode `potentialPremiumProfit` qui double le gain pour ces plantes.

```
double potentialPremiumProfit()
```

Q3.5. Réalisez un `commit` associé au message : "tp00-07::exo-horticulture". Soumettez ensuite l'ensemble des `commit` au dépôt.